

seL4

Page Content

- [General](#)
 - [Licenses](#)
 - [Repositories](#)
 - [Documentation](#)
- [seL4 ports](#)
 - [ARM](#)
 - [General](#)
 - [Xilinx Zynq Ultrascale](#)
 - [RasPi3](#)
 - [RISC-V](#)
 - [QEMU/Spike](#)
- [Pitfalls](#)
- [seL4 libraries](#)
- [seL4 OS projects](#)
 - [random collection](#)
- [Debugging](#)
- [Logging in seL4](#)
- [Formal Verification](#)
 - [General](#)
 - [Verification aspects of seL4 code base](#)
- [Attachments](#)

Sub-Pages

- [CAMkES Notes and FAQ](#)
- [seL4/CAMkES Reference](#)
- [seL4 abstract specification documentation](#)
- [seL4 as ARM TrustZone SWd OS](#)
- [seL4 boot process](#)
- [seL4 Camkes VM Examples](#)
- [seL4 Core and seL4 Core Platform \(sel4cp\)](#)
- [seL4 DARPA projects](#)
- [seL4 Device Driver Framework](#)
- [seL4 docker build environment](#)
- [seL4 elfloader](#)
- [seL4 printf code path](#)
- [seL4 Self Test Suite](#)
- [seL4 Timer Drivers and TimeServer](#)
- [seL4 - using the proof](#)

General

- <https://sel4.systems>
- Data61, Trustworthy Systems: <https://ts.data61.csiro.au/projects/TS>

Licenses

- seL4 Kernel: GPLv2 (set 07/2014), Rechte liegen bei General Dynamics C4 Systems, proprietäre Lizenz möglich
- Bibliotheken, Userland-Tools: BSD-Lizenz
- seL4 Historie
 - ursprünglich entwickelt an der University of New South Wales
 - Weiterentwicklung bei NICTA seit 2004
 - NICTA Ausgründung Open Kernel Labs, Weiterentwicklung als OKL4, Übernahme 09/2012 durch General Dynamics C4 Systems

Repositories

- code owned by seL4 Foundation (and formerly General Dynamic C4 Systems): <https://github.com/SEL4>
- code owned by Data61 was created in <https://github.com/SEL4PROJ>, it has been moved to the seL4 Foundation repos now also, see also

[SEOS-1960](#) - Getting issue details...

STATUS

Documentation

- Status und Roadmap: <https://sel4.systems/Info/Roadmap/home.pml>
- latest manual: <http://sel4.systems/Info/Docs/seL4-manual-latest.pdf>

- <https://wiki.sel4.systems/Getting%20started>
- UNSW course COMP9242 "Advanced Operating Systems"
 - Material: <https://www.cse.unsw.edu.au/~cs9242/current/>
 - Lectures: <https://www.youtube.com/playlist?list=PLbSaCpDlfd6qLbEsKquVo3--0gwYBmrUV>
- Presentations
 - see also [Attachments](#) below
 - see also [seL4 Summits](#)
 - see also seL4 Foundation Youtube channel: <https://www.youtube.com/channel/UCIqVY1XFPOPIZ1z2A9TrmMA/about>
 - Flying autonomous aircraft: Mixed-criticality support in seL4: <https://www.youtube.com/watch?v=ijTTZgQ8cB4>
 - seL4 is free and open source: <https://www.youtube.com/watch?v=heSmrHzHcuM>
 - fast IPC on L4: https://www.youtube.com/watch?v=mRr1ICJse_I
 - LCA 2020: Verified seL4 on secure RISC-V processors, <https://www.youtube.com/watch?v=wJ96s3pNtI0>
- TRENTOS
 - see also [2018-09-27 Training](#)
 - [CAMkES Notes and FAQ](#)
 - [TRENTOS Handbook](#)
 - [Deprecated - TRENTOS-M Teaching Material](#)

seL4 ports

ARM

General

- Status
 - working
 - ARMv7: core functions, virtualisation, multi core
 - ARMv8 AARCH32 is similar to ARMv7, so things should work
 - ARMv8 AARCH64: core functions
 - missing:
 - ARMv8 AARCH64: virtualization, multi core
- specific features
 - virtualization
 - requires the ARM Virtualization Extension (missing on Cortex-A8/A9)
 - https://genode.org/documentation/articles/arm_virtualization
 - TrustZone
 - could run a dummy seL4 to lock it down
 - see also [seL4 as ARM TrustZone SWd OS](#)

Xilinx Zynq Ultrascale

- see also [Xilinx](#)
- <https://dornerworks.com/blog/hardware-security-growing-concern-sel4-ecosystem-offers-one-solution>
- <https://dornerworks.com/news/new-commits-to-open-source-sel4>

RasPi3

- see [Raspberry Pi](#)

RISC-V

- docs
 - <https://docs.sel4.systems/Hardware/RISCV.html>
 - <https://www.slideshare.net/HeshamALMatary/sel4-on-riscvlowrisc-orconf15>
 - <http://heshamelmatory.blogspot.de/2015/06/sel4-on-risc-v-is-running-sos-simple.html>
- notes
 - see also
 - [seL4 boot process](#)
 - [seL4 elfloader](#)
 - [RISC-V BBL \(Proxy Kernel\)](#)
 - [seL4 integration on HCSC1 FPGA or chip](#)
 - seL4 support BBL/ProxyKernel or OpenSBI (<https://github.com/riscv/opensbi>)
 - Run seL4 in M-Mode or S-Mode?
 - S-Mode (default)
 - seL4 in S-Mode avoids running anything at all in M-Mode
 - platform boots in M-Mode
 - our bootstrap code there sets up things for seL4 in S-Mode
 - jump to S-Mode and run seL4
 - never need to ever enter M-Mode again.
 - M-Mode calls should cause platform halt, but seL4 in S-Mode shall never calls M-Mode anyway
 - M-Mode
 - technically possible

- what do we gain from moving seL4 fully to M-Mode?
 - lock down M Mode
 - S-Mode can be used for "virtualization"
- avoid any firmware code and SBI support
 - not needed anyway if we implement everything in seL4
 - why should our chip make use M-Mode firmware at all?
 - Nice approach, but not too practical if we control seL4 internals and drivers anyway
 - is there anything that really must run in M-Mode? Seems not, but may depend on the chip
 - Who creates and verifies M-Mode Firmware?
- M-Mode does not support MMU, could limit accessible memory on a 32-bit system but not an issue on 64-bit system
- read/write access to all of the lower privilege registers - but are there any registers that need to be changed at runtime?

QEMU/Spike

- DRAM starts at 0x80000000
- first 2 MiB (0x80000000 - 0x80200000) are reserved for SBI
 - BBL and OpenSBI configures PMP to prevent S-Mode access the parts of this area that they use
 - pmpaddr0: 0x20000000 (= &_ftext = 0x80000000)
 - pmpaddr1: 0x20004c00 (= &_end = 0x80013000)
 - pmpaddr2: 0xffffffff
 - pmpcfg0: 001f0800
- ElfLoader and seL4 can't run from 80000000, but must move a bit
 - ElfLoader runs from 0x80200000
 - seL4 kernel runs from 0x84200000 (=DRAM start + 66 MiB)
- sel4test msut set Sel4testHaveTimer=OFF, since QEMU/spike does not emulate a timer peripheral

Pitfalls

- there is no notification for revoked capabilities
 - avoids side channels
 - if needed, the OS must implement a notification mechanism when revoking caps.
 - at least during development, return codes for syscalls on caps must always be checked, so one can learn early if something is broken or mis-configured
- for IPC send there is no return code at all, so one may never learn if or why it failed.
 - this avoids any side channels where the caller could learn thing about the endpoint that it is not supposed to learn
 - in some cases (during development) the missing return codes is really a pain, as some error related to a broken or mis-configured system could be really helpful
- endpoint badges can use bit 0-27 only, kernel will silently ignore any usage of bit 28-31.
- seL4's untyped memory list is not prepared for a scenario, where a physical address space exists that is bigger than the virtual address space
 - Obviously, with MMU of (e.g. during boot), only the lower part of the physical address space is accessible anyway
 - with ARM LPAE enabled, the MMU tables hold 64-bit phys addresses even in AARCH32 with a 32bit OS is running
 - An example for such a platform is ARM's Juno Board, which has 8 GiB DRAM starting at the phys address 2 GiB. In seL4, just the lowest 2 GiB can be used as everything beyond can't be represented in the untyped memory list
- only pages can be shared, not page tables. Page caps must be duplicated to share a page
- on ARM AARCH64 seL4 in default configuration will crash on boot if started in EL2 or EL3
 - the kernel loader activates the EL1 MMU. Since the EL2/EL3 MMU is still disabled, thing crash when the loader jumps to a virtual addresses 0xffffffff8000000000 to start seL4

seL4 libraries

- **libsel4**: seL4 system library, wraps syscalls, object definitions ...
- **libmuslc**: compact standard C library, based on <http://www.musl-libc.org>
- **libsel4muslcsys**: minimal libmuslc for root task
- **libplatsupport**: platform specific stuff
- **libsel4platsupport**: for bootstrapping
- **libcpio**: process cpio archive to find user apps (usually in ELF format)
- **libelf**: process and load ELF binaries
- **libsel4cspace**: CSpace utils
- ... and some more

seL4 OS projects

- OS from the seL4 teaching courses
 - <http://www.cse.unsw.edu.au/~cs9242/16/project/design.shtml>
 - https://www.cs.hs-rm.de/~kaiser/1717_aos.php
- selkie: <https://github.com/celskeggs/selkie>
- Genode OS Framework
- Qubes OS: [Qubes OS](https://www.qubes-os.org) <https://www.qubes-os.org> (Gernot: "We have made some investigations on this in the past, and there's talk about third-party funding for [porting it on top of seL4], but nothing concrete yet.")
- UX/RT: <https://gitlab.com/uxrt>

random collection

- <http://sel4.org/Info/GettingStarted/DebianToolChain.pml>
- <https://cse.jashankj.space/17s2/COMP9242/lectures/2017-07-28/>
- <https://dornerworks.com/tag/sel4>
- <https://dornerworks.com/blog/build-http-webpage-ui-using-sel4-bring-verified-security-iot-projects>
- QEMU-x86: <https://wiki.sel4.systems/Hardware/Qemu>
- DARPA Projekt
 - <http://loonwerks.com/projects/hacms.html>
 - <https://github.com/smaccm/smaccm/releases>
- Rump Kernels
 - <https://research.csiro.au/tsblog/using-rump-kernels-to-run-unmodified-netbsd-drivers-on-sel4>
 - <http://www.fixup.fi/misc/rumpkernel-book>
- Rust on sel4:
 - <https://github.com/PolySync/cargo-fel4.git>
 - <https://github.com/celskeggs/searust>

Debugging

- <https://github.com/sel4/libsel4debug>
- Debugging Guide: <http://www.cse.unsw.edu.au/~cs9242/16/project/debugging.shtml>
- sel4 Wiki:
 - Debugging guide: <https://wiki.sel4.systems/Debugging%20guide>
- Debugging userspace threads: <https://wiki.sel4.systems/Debugging%20userspace%20threads>
- Debugging sel4 with GDB: <https://sel4.systems/Community/Devdays/Workshop2016/chris-sel4ws2016.pdf>

Logging in sel4

- see also [sel4 printf code path](#)
- there is no concept of a log level or verbosity in the kernel and LibUtils is not used at all. Thus playing with `LibUtilsDefaultZfLogLevel` is useless for kernel output
- applications can use `sel4_DebugPutChar()` that will issue a `SysDebugPutChar` syscall that calls `kernel_putchar()`
- Defines and related CMake config options
 - **CONFIG_DEBUG_BUILD** (CMake: **KernelDebugBuild**, enabled by default)
 - the printing function really do something and are not `ifdef'd` to void: `kernel_putchar()`, `printf()`, `puts()`
 - **CONFIG_PRINTING** (CMake: **KernelPrinting**, defaults to setting of **KernelDebugBuild**)
 - print some more kernel internals
 - enables syscall `SysDebugPutChar`
 - **CONFIG_COLOUR_PRINTING** (CMake: **KernelColourPrinting**, enabled by default)
 - use ANSI escape sequence to add some colors, should be disabled when logging to a file

Formal Verification

General

- <http://sel4.systems/Info/FAQ/proof.pml>
- see also [Formal Verification](#)
- nur TCB muss verifiziert sein (und nicht alles, was auf dem Kern läuft)
- nur C-Code kann momentan verifiziert werden
 - code must gut strukturiert sein.
 - ungeeignet sind C++, Haskell, weil ...
 - unklare Semantik (C++)
 - automatischer Garbage Collector schwierig
 - Sprache und Laufzeitsystem zu groß
- Workflow
 - [2014 - Affordable trustworthy-systems \(Slides\).pdf](#)
 - [2014 - Filesystems deserve verification too \(Slides\).pdf](#)
- Binärverifikation
 - benötigt nicht nur C Code, sondern auch das formale Modell
 - Ablauf: [2018 - Verification of GCC-generated sel4 binary.pdf](#)
 - Status
 - ARM: ja
 - RISC-V: in Arbeit, benötigt formale ISA Spec
 - x86: nein
 - <https://entropy2018.sciencesconf.org/data/myreen.pdf>
 - <https://dornerworks.com/news/modular-build-binary-verification-sel4>
- Presentations
 - CPP'22: The sel4 verification: the art and craft of proof and the reality of commercial support: <https://www.youtube.com/watch?v=QL5oGMxpmJI>

Verification aspects of seL4 code base

- Observation: there are commits in the seL4 repo which introduce changes solely to ease verification (= no changes in functionality)
- Quote: "This formal verification was only feasible due to the non-preemptible design of seL4." from To Preempt or Not To Preempt, That Is the Question; Bernard Blackham, Vernon Tang and Gernot Heiser NICTA and University of New South Wales, Sydney, Australia {Bernard.Blackham, Vernon.Tang, gernot}@nicta.com.au
- Things which help with verification (according to commit messages)
 - avoid implicit casts
 - reorder expressions (theory: code path can finish earlier = fewer variables are "used" by the code path)
 - use shift rather than divide operator

Attachments

File	Modified
PDF File 2014 - Filesystems deserve verification too (Slides).pdf	Sep 21, 2018 by HG123912
PDF File 2014 - Affordable trustworthy-systems (Slides).pdf	Sep 21, 2018 by HG123912
PDF File 2018 - Verification of GCC-generated seL4 binary.pdf	Sep 21, 2018 by HG123912

Drag and drop to upload or [browse for files](#)
[Download All](#)